XAUUSD Strategy Lab

PRODUCT REQUIREMENTS AND TECHNICAL SPECIFICATION

v1.0 - Development handoff

Prepared for implementation by Codex

23 August 2026

Core decision: build a reusable local Strategy Lab with shared data, charting and backtesting infrastructure. Strategies are plug-in modules. This is not a TradingView clone and not a separate application per strategy.

1. Executive brief

The product is a single-user local web application for importing candle data, switching timeframes, viewing indicators on an interactive candlestick chart, configuring strategy parameters, running deterministic backtests and inspecting results. The first included strategy is Bollinger Awesome for XAUUSD. The architecture must allow later strategies to reuse the same data, charting and execution engines.

Target architecture at a glance

1.1 Product goals

- Import user-provided candle files and validate them before use.
- Aggregate valid lower-timeframe candles into higher timeframes.
- Display responsive candlesticks, indicator overlays, oscillator panes and trade markers.
- Run transparent, deterministic and reproducible backtests without lookahead bias.
- Compare strategy configurations under identical execution assumptions.
- Persist datasets, strategy configurations and backtest runs locally.
- Create a clean boundary for optional AI analysis later without making AI part of v1 execution.

1.2 Non-goals for v1

- No real-time market feed, broker connection or live order execution.
- No TradingView-compatible scripting language or arbitrary user code editor.
- No drawing tools, multi-chart layouts, social features or alert service.
- No authentication, multi-user permissions or cloud collaboration.
- No automatic AI strategy optimization or parameter search.
- No claim that a backtest proves future profitability.

Priority order: execution correctness first, reproducibility second, usability third, visual polish fourth. A beautiful but incorrect backtester is a failed product.

1.3 Primary user journey

Step	User action	System response
1	Import candle file	Validate schema, timestamps, OHLC integrity, gaps and duplicates.
2	Choose dataset and timeframe	Load or generate cached candles for the selected timeframe.
3	Choose strategy and parameters	Display calculated indicators and qualified signals.
4	Run backtest	Execute deterministic simulation and persist a versioned run.
5	Review results	Show metrics, equity, drawdown, trades and chart-linked markers.
6	Compare or export	Compare saved runs or export metrics and trades.

2. Functional requirements

2.1 Dataset import and validation

ID	Requirement	Acceptance criterion
DATA-01	Accept CSV and Parquet candle files.	A valid file can be imported without editing source code.
DATA-02	Require timestamp, open, high, low and close. Volume is optional.	Missing required columns block import with a specific error.
DATA-03	Collect source metadata: instrument, provider, timezone and quote type.	Metadata is stored with the dataset and shown in the UI.
DATA-04	Normalize timestamps to UTC while preserving source timezone metadata.	The same instant is rendered correctly in UTC, New York and Jakarta.
DATA-05	Sort records, detect duplicates, invalid OHLC values and missing intervals.	Import report lists counts, examples and actions taken.
DATA-06	Do not silently repair material errors.	User must explicitly approve safe duplicate removal; invalid OHLC rows block import.
DATA-07	Store normalized data as Parquet under a stable dataset ID.	Reopening the application does not require re-import.

2.2 Canonical candle contract

Field	Type	Required	Rule
timestamp	UTC datetime	Yes	Unique and ascending after normalization.
open	float64	Yes	Finite and greater than zero.
high	float64	Yes	At least max(open, close, low).
low	float64	Yes	At most min(open, close, high).
close	float64	Yes	Finite and greater than zero.
volume	float64/null	No	Non-negative when supplied.

Dataset metadata must include: dataset_id, instrument, provider, quote_type (bid, ask, midpoint or unknown), source_timezone, base_timeframe, imported_at, row_count, first_timestamp, last_timestamp, validation_summary and source_file_hash.

2.3 Timeframe aggregation

- Only aggregate upward. A 15-minute source cannot create valid 5-minute candles.
- Default target timeframes: 1m, 5m, 15m, 30m, 1h and 4h, limited to valid multiples of the base timeframe.
- OHLC aggregation: first open, maximum high, minimum low and last close. Volume is summed when available.
- Use UTC-aligned windows. Keep session timezone logic separate from storage and aggregation.
- Exclude an incomplete trailing aggregate candle by default; provide an explicit setting to include it for inspection only.
- Cache generated timeframes and invalidate caches when the source dataset changes.
- Surface gap counts and do not fabricate missing base candles.

2.4 Charting

ID	Requirement
CHART-01	Interactive candlestick chart with zoom, pan, crosshair and responsive resize.
CHART-02	Timeframe switcher using available valid aggregations.
CHART-03	Price-pane overlays for SMA, EMA, Bollinger Bands, stop and target levels.
CHART-04	Separate synchronized pane for Awesome Oscillator.
CHART-05	Markers for signal, entry, exit, stop, target, session exit and AO exit.
CHART-06	Crosshair legend showing time, OHLC and active indicator values.
CHART-07	Clicking a trade row scrolls and zooms the chart to that trade.
CHART-08	Overlay visibility toggles without rerunning the backtest.
CHART-09	Loading, empty-data and error states must be explicit.

3. Strategy framework

Strategies must be plug-in modules executed by the shared engine. A strategy module calculates indicators and signals but must not contain chart rendering, file import, result formatting or persistence logic.

3.1 Required strategy interface

```
class Strategy:
key: str
name: str
parameter_schema: dict

def validate_parameters(self, params) -> list[ValidationError]: ...
def calculate_indicators(self, candles, params) -> DataFrame: ...
def generate_signals(self, frame, params) -> DataFrame: ...

# Signal output columns
signal_long: bool
signal_short: bool
exit_long: bool
exit_short: bool
signal_reason: str
```

Strategy modules produce deterministic indicator and signal columns. The backtest engine alone decides fills, position state, costs, risk sizing and trade outcomes.

3.2 Included strategy: Bollinger Awesome v1

Parameter	Default	Validation / notes
Basis type	SMA	SMA or EMA
Bollinger length	20	Integer >= 2
Bollinger deviation	2.0	Float > 0
Fast EMA length	3	Integer >= 2
AO fast / slow	5 / 34	Positive integers; fast < slow
Close-inside-BB filter	On	On or off
Squeeze filter	On	On or off
Squeeze length / threshold	100 / 50	Positive integer / positive float
AO confirmation	Direction only	Off, direction, zero cross, proximity
AO zero-cross lookback	3 bars	0-20
AO proximity	25% of 100-bar max	1-100%; normalization >= 20
Session	03:00-12:00 New York	Enable/disable; IANA timezone

3.3 Bollinger Awesome entry logic

- Long base signal: EMA(3) crosses above the Bollinger basis and the signal candle closes above the basis.
- Short base signal: EMA(3) crosses below the basis and the signal candle closes below the basis.
- Direction-only AO: long requires AO > prior AO; short requires AO < prior AO.
- Zero-cross AO: direction condition plus a directional zero cross within the configured lookback.
- Proximity AO: direction condition plus abs(AO) <= configured percentage of the rolling maximum abs(AO).

- Close-inside-BB: long close < upper band; short close > lower band.
- Squeeze score: current BB width / SMA(BB width, squeeze length) x 100. Qualified when score > threshold.
- A signal is evaluated only on a completed candle. The signal is not itself a fill.

3.4 Backtest configuration

Group	Configurable controls
Window	Start/end timestamps or last N candles; long, short or both.
Stops	ATR length and ATR stop multiplier.
Targets	Reward-to-risk target; fixed, AO reversal, or whichever occurs first.
Sizing	Risk percent or fixed quantity; point value, quantity step and minimum quantity.
Costs	Spread, commission and slippage with clearly stated units.
Session	Enable, session string, IANA timezone and force-exit toggle.
Collision	Conservative stop-first or lower-timeframe replay when available.

4. Backtesting semantics

These rules are contractual. A change to any execution rule creates a new engine version and must be visible in saved run metadata.

ID	Rule
EXEC-01	Signals use only information available at the completed signal candle. No future rows, centered windows or backfilled future values.
EXEC-02	A market entry signal fills at the next available candle open, adjusted by configured spread/slippage.
EXEC-03	Only one position is allowed. Pyramiding and overlapping positions are disabled in v1.
EXEC-04	Stop and target distances are frozen from the entry setup and do not drift with later ATR values.
EXEC-05	Protective stop and target become active immediately after entry fill.
EXEC-06	If stop and target are both inside one OHLC candle and no lower-timeframe path exists, count the stop first.
EXEC-07	When valid lower-timeframe data exists, optional replay may determine which level was touched first. Record whether replay was used.
EXEC-08	AO reversal exits occur after the completed candle that confirms the reversal and fill at the next available open.
EXEC-09	Force-session exit occurs at the next available executable price after the session ends and includes costs.
EXEC-10	No new entry may fill outside the configured test window or session.
EXEC-11	Every run stores the dataset hash, engine version, strategy version and full parameter JSON.
EXEC-12	Repeated runs with identical inputs must produce identical trades and metrics.

4.1 Cost model

- Define whether supplied candles are bid, ask or midpoint. Unknown quote type must remain visible as a data-quality limitation.
- For midpoint candles, apply half-spread adversely at entry and exit. Long entry pays ask; long exit receives bid. Reverse for shorts.
- Slippage is always adverse and may be configured in price units or ticks, but the selected unit must be stored.
- Commission may be per quantity per side or a flat per-side amount. Do not mix models silently.
- Net P&L; equals gross price P&L; minus spread, slippage and commission.

4.2 Position sizing

Risk-based quantity = floor_to_step((equity x risk_percent) / (stop_distance x point_value)). If the computed quantity is below the minimum quantity, the default behavior is to skip the trade and record the reason. Do not force the minimum quantity because that can exceed the requested risk.

For comparison tests, fixed quantity is allowed. Metrics in R must use the actual initial monetary risk of each filled trade, including the final rounded quantity.

4.3 Determinism and versioning

- Assign semantic versions to the engine and each strategy module.
- Generate a run fingerprint from dataset hash, timeframe, engine version, strategy version and canonical parameter JSON.
- Never overwrite a historical run when code or parameters change. Create a new run.
- Allow users to duplicate a configuration, but preserve its parent run reference.

5. Results and analysis

Metric	Definition / calculation
Closed trades	Total completed trades; also split long and short.
Net profit	Gross P&L; minus all modeled costs.
Win rate	Winning closed trades / all closed trades.
Profit factor	Gross winning P&L; / absolute gross losing P&L.;
Expectancy in R	Mean net trade result divided by initial risk per trade.
Average win/loss	Display in currency, percent and R where defined.
Maximum drawdown	Peak-to-trough equity decline in currency and percent.
Loss streak	Maximum consecutive losing trades.
Exposure	Time in market / test-window time.
Duration	Average and median bars and elapsed time per trade.
Long/short	Independent trade count, win rate, PF, expectancy and net result.

5.1 Required result views

View	Contents
Summary	Key metric cards, configuration identity and data-quality warnings.
Equity	Equity curve, drawdown curve and optional buy-and-hold reference.
Trades	Sortable, filterable trade table linked to the chart.
Chart	Candles, indicators, signals, entries, exits, stops and targets.
Long/short	Side-by-side performance breakdown.
Run details	Dataset metadata, cost assumptions, versions and full parameters.
Comparison	Two to four saved runs compared on the same metric definitions.

5.2 Trade-log contract

- trade_id, run_id, direction and quantity
- signal_time, signal_price, entry_time and entry_price
- initial_stop, initial_target and initial_risk_cash
- exit_time, exit_price and exit_reason
- gross_pnl, spread_cost, slippage_cost, commission_cost and net_pnl
- result_r, holding_bars, holding_seconds, MAE and MFE
- signal_reason and any skipped/exception flags

5.3 Export

- Export a run summary as JSON.
- Export the trade log as CSV.
- Export configuration as JSON that can be re-imported.
- Exported data must include dataset, strategy and engine identifiers.

6. Technical architecture

Layer	Choice	Responsibility
Frontend	React or Next.js + TypeScript	Local web UI, controls, result views and chart coordination.
Chart	TradingView Lightweight Charts	Candles, overlays, panes, markers, crosshair and navigation.
API	Python 3.12 + FastAPI	Typed local API boundary and validation.
Data	Polars + Parquet	Validation, aggregation, indicators and cached candle storage.
Persistence	SQLite	Dataset metadata, configurations, runs and trade summaries.
Tests	pytest + frontend test runner	Engine unit tests, API integration tests and critical UI behavior.

The frontend must not recalculate trading logic. It renders API results. Python is the source of truth for candles, indicators, signals, fills, costs and metrics.

6.1 Suggested repository structure

```
strategy-lab/
frontend/
src/components/chart/
src/components/backtest/
src/pages/
src/api/
backend/
app/api/
app/data/
app/indicators/
app/strategies/
app/backtest/
app/models/
app/storage/
data/raw/
data/processed/
tests/fixtures/
docs/
AGENTS.md
README.md
```

6.2 Minimum local API

Method	Path	Purpose
POST	/api/datasets/import	Import, validate and persist a dataset.
GET	/api/datasets	List datasets and validation summaries.
GET	/api/datasets/{id}/candles	Return paged/limited candles for timeframe and date range.
GET	/api/strategies	List strategies and parameter schemas.
POST	/api/backtests	Run and persist a backtest.
GET	/api/backtests/{run_id}	Return configuration, metrics and curves.
GET	/api/backtests/{run_id}/trades	Return filterable trade records.
POST	/api/backtests/compare	Compare selected compatible runs.

7. User interface requirements

7.1 Primary layout

Region	Required content
Top bar	Dataset selector, timeframe selector, strategy selector and Run Backtest button.
Left panel	Grouped strategy, session, risk, cost and display controls.
Main area	Candlestick chart with synchronized indicator panes.
Bottom panel	Summary, equity, drawdown, trades, long/short and run-detail tabs.
Right/overlay	Optional compact legend and active trade information.

7.2 Interaction requirements

- Changing display-only settings must not rerun the backtest.
- Changing strategy or execution settings marks displayed results as stale until rerun.
- The Run button is disabled when validation errors exist.
- Every result view displays the run ID and configuration name.
- Clicking a chart marker selects its trade row; clicking a trade row centers the chart.
- Large datasets must be windowed for chart rendering; never send millions of points to the browser at once.
- Use clear warnings for unknown quote type, gaps, insufficient warm-up and collision assumptions.

7.3 Baseline configuration

Setting	Baseline
Instrument/timeframe	XAUUSD / 15-minute standard candles
Bollinger	SMA 20, deviation 2.0
Fast EMA	3
AO	5/34, direction only
Filters	Close-inside-BB on; squeeze 100/50 on
Session	03:00-12:00 America/New_York, weekdays
Stop/target	ATR(14) x 1.0 stop; target 1.5R
Sizing	0.5% equity risk
Collision policy	Conservative stop-first

8. Testing and acceptance

8.1 Mandatory engine tests

ID	Test
TEST-01	Aggregation fixture produces exact expected OHLCV for every supported timeframe.
TEST-02	A future price modification does not alter any earlier indicator or signal.
TEST-03	Signal at bar close fills at the next bar open, never the signal close.
TEST-04	Stop-only, target-only and neither-touched fixtures close correctly.
TEST-05	Same-candle stop and target counts stop first without replay.
TEST-06	Lower-timeframe replay chooses the actual first touch in controlled fixtures.
TEST-07	Long and short cost calculations are adverse and symmetrical.
TEST-08	Risk rounding never exceeds requested risk by forcing minimum quantity.
TEST-09	New York sessions behave correctly across daylight-saving transitions.
TEST-10	Identical run fingerprints produce identical trades and metrics.
TEST-11	Metric fixtures match independently calculated expected values.
TEST-12	Bollinger Awesome indicators match trusted reference vectors.

8.2 Acceptance criteria

- A user can import a valid candle file and see a validation report without editing code.
- A user can switch among valid higher timeframes and see correctly aggregated candles.
- The Bollinger Awesome indicators and signals render on synchronized chart panes.
- A baseline backtest produces persisted trades, metrics, equity and drawdown.
- A trade row and its chart markers navigate to each other.
- A saved run can be reopened after application restart with unchanged results.
- Two compatible runs can be compared without recalculating historical runs.
- All mandatory engine tests pass locally in one documented command.
- README documents installation, startup, data schema and first-run workflow.
- No v1 feature requires an OpenAI API key.

9. Implementation plan

Milestone	Description	Exit gate
M0 - Bootstrap	Monorepo, local scripts, linting, tests, README skeleton.	Frontend and backend start with one command each.
M1 - Data	Import, validation, UTC normalization, Parquet storage, aggregation.	DATA requirements and aggregation fixtures pass.
M2 - Engine	Positions, fills, stops, targets, costs, sizing, metrics and persistence.	Execution tests pass before UI integration.
M3 - Strategy	Plug-in interface and Bollinger Awesome module.	Reference indicator and signal fixtures pass.
M4 - API	Dataset, candle, strategy and backtest endpoints.	API integration tests pass.
M5 - UI	Controls, chart, AO pane, markers and result tabs.	Primary journey works end to end.
M6 - Compare/export	Saved runs, comparison, CSV/JSON export.	Run identity remains reproducible.
M7 - QA	Performance pass, error states, documentation and clean install test.	Definition of done is satisfied.

9.1 Development sequencing rules

- Do not start chart polish before the engine fixtures pass.
- Do not implement AI, live data or broker connectivity during v1.
- Use small, reviewable commits aligned to one milestone.
- Add tests in the same change as execution logic.
- Record any deliberate deviation from this PRD in docs/decisions.md.
- If a requirement is ambiguous, stop and ask rather than choosing a favorable backtest assumption.

9.2 Codex handoff instructions

Codex should first inspect this PRD, propose the repository plan and identify unresolved data assumptions. It should then implement milestone by milestone, run tests after every milestone and report exact files changed, commands run and remaining risks. It must not silently expand scope.

Suggested first Codex prompt

```
Read XAUUSD-Strategy-Lab-PRD-v1.0.pdf completely.
Build the local Strategy Lab described in it.

Before writing code:
1. Summarize the architecture and execution semantics.
2. List assumptions that still require my decision.
3. Propose a milestone plan mapped to the PRD IDs.
4. Inspect the repository and preserve unrelated work.

Then implement only M0 and M1. Add tests, run them, and stop for review.
Do not begin later milestones until I approve the previous milestone.
```

9.3 Definition of done

- Clean installation from README succeeds on a fresh local machine.
- All mandatory tests pass.
- Baseline XAUUSD workflow runs end to end with a provided valid dataset.
- Backtest assumptions are visible in the UI and saved with each run.
- There are no known lookahead paths or silent data repairs.
- The app remains usable without network access after dependencies are installed.
- A second sample strategy can be added without modifying the shared chart or engine contract.

10. Later roadmap - explicitly outside v1

v1.1	Batch parameter sweeps and walk-forward analysis.	Stable deterministic engine and run comparison.
v1.2	AI review of compact backtest summaries.	Versioned JSON summary contract and cost controls.
v1.3	Paper-trading replay from appended candles.	Incremental engine state and reliable feed adapter.
v2	Real-time feed, alerts and optional broker integration.	Security, reconciliation and failure-handling design.

AI must receive compact structured outputs - metrics, regime breakdowns, configuration, selected trades and diagnostics - rather than raw candle history. The deterministic engine remains authoritative. AI may recommend a test but must not rewrite past results or silently change execution assumptions.

11. Technical references

- TradingView Lightweight Charts: https://tradingview.github.io/lightweight-charts/
- Lightweight Charts panes: https://tradingview.github.io/lightweight-charts/tutorials/how_to/panes
- FastAPI: https://fastapi.tiangolo.com/
- Polars time-series resampling: https://docs.pola.rs/user-guide/transformations/time-series/resampling/

Final scope: a local, reusable Strategy Lab. One shared data engine, one shared backtest engine, one shared chart and results interface, and versioned strategy modules beginning with Bollinger Awesome for XAUUSD.